Схема:

- `flight(flight_nr, dept, dest)`
- `plane(flight_nr, plane_nr)`
- `type(plane_nr, type)`

Запросы к базе рейсов

(a) Типы самолётов, используемых на рейсах из BRU

SQL:

```
SELECT DISTINCT t.type
FROM flight f
JOIN plane p ON f.flight_nr = p.flight_nr
JOIN type t ON p.plane_nr = t.plane_nr
WHERE f.dept = 'BRU';
```

Реляционная алгебра:

```
π_type(σ_dept='BRU'(flight) ⋈ plane ⋈ type)
```

(b) Самолёты, не участвующие ни в одном рейсе

Отношение `type` содержит все самолёты. Отношение `plane` — только те, что задействованы в рейсах.

SQL:

```
SELECT plane_nr FROM type
EXCEPT
SELECT plane_nr FROM plane;
```

Реляционная алгебра:

```
π_plane_nr(type) − π_plane_nr(plane)
```

(c) Типы самолётов, задействованных во всех рейсах

Тип T участвует во всех рейсах — значит не существует рейса, на котором не было бы ни одного самолёта типа T.

SQL:

```
SELECT DISTINCT t.type
FROM type t
WHERE NOT EXISTS (
    SELECT 1 FROM flight f
    WHERE NOT EXISTS (
        SELECT 1
        FROM plane p
        JOIN type t2 ON p.plane_nr = t2.plane_nr
        WHERE p.flight_nr = f.flight_nr
          AND t2.type = t.type
    )
);
```

(d) Маршруты (dept, dest), для которых используются не все типы самолётов

Маршрут не использует все типы — значит существует тип, которого на этом маршруте нет.

SQL:

```
SELECT DISTINCT f.dept, f.dest
FROM flight f
WHERE EXISTS (
    SELECT 1 FROM type t
    WHERE NOT EXISTS (
        SELECT 1
        FROM plane p
        JOIN flight f2 ON p.flight_nr = f2.flight_nr
        JOIN type t2 ON p.plane_nr = t2.plane_nr
        WHERE f2.dept = f.dept
          AND f2.dest = f.dest
```

```
            AND t2.type = t.type
        )
    );
```

(e) Номера рейсов, встречающихся несколько раз

Рейс встречается несколько раз, если в `flight` есть более одной строки с этим `flight_nr`.

SQL:

```
SELECT flight_nr
FROM flight
GROUP BY flight_nr
HAVING COUNT(*) > 1;
```

Реляционная алгебра (через самосоединение):

```
π_flight_nr(σ_dest1≠dest2 ∨ dept1≠dept2 (
    ρ_{flight_nr, dept1, dest1}(flight) ◃▹_flight_nr
ρ_{flight_nr, dept2, dest2}(flight)
))
```

(f) Пары различных номеров рейсов на одном маршруте

SQL:

```
SELECT f1.flight_nr AS nr1, f2.flight_nr AS nr2
FROM flight f1
JOIN flight f2
  ON f1.dept = f2.dept
 AND f1.dest = f2.dest
 AND f1.flight_nr < f2.flight_nr;
```

Условие `f1.flight_nr < f2.flight_nr` исключает симметричные дубли и пары с самим собой.

(g) Самолёты, чей тип используется хотя бы раз на каждом маршруте

Самолёт p удовлетворяет условию, если его тип T покрывает все маршруты, то есть не существует маршрута (dept, dest), на котором нет ни одного самолёта типа T.

SQL:

```
SELECT DISTINCT p.plane_nr
FROM plane p
JOIN type t ON p.plane_nr = t.plane_nr
WHERE NOT EXISTS (
    SELECT 1 FROM flight f
    WHERE NOT EXISTS (
        SELECT 1
        FROM plane p2
        JOIN flight f2 ON p2.flight_nr = f2.flight_nr
        JOIN type t2 ON p2.plane_nr = t2.plane_nr
        WHERE f2.dept = f.dept
          AND f2.dest = f.dest
          AND t2.type = t.type
    )
);
```

Истина или ложь?

(a) Каждое выражение реляционной алгебры без отрицания монотонно

Истина.

Оператор называется монотонным, если добавление кортежей во входные отношения не может уменьшить результат. Все операторы без отрицания (проекция, селекция с положительными условиями, декартово произведение, соединение, объединение, пересечение) монотонны:

- Проекция и соединение: добавляем кортежи — могут появиться новые результаты, но старые не исчезнут.
- Пересечение тоже монотонно: если кортеж $t \in R \cap S$, и $R \subseteq R'$, $S \subseteq S'$, то $t \in R' \cap S'$.

Отрицание (разность –) — единственный немонотонный оператор: добавление кортежа в правую часть может убрать его из результата.

(b) Операторы ∩ и ∪ избыточны

Ложь.

- **∩ избыточно:** `A ∩ B = A − (A − B)`. Пересечение выражается через разность.
- **∪ не избыточно:** объединение нельзя выразить через проекцию, селекцию, декартово произведение и разность. Это фундаментальный результат: без ∪ невозможно создать выражение, результат которого содержит кортежи из двух не связанных частей. Формально, все выражения без ∪ выдают результат, вложенный в декартово произведение входных отношений, что не позволяет смешивать кортежи из несвязанных источников. ∩ избыточно, ∪ — нет. Утверждение, что оба избыточны, ложно.

Запросы к графу

Схема: `G(A, B)` — направленный граф (ребро A → B).

(a) Узлы без входящих рёбер

Узлы без входящих рёбер — это узлы, которые никогда не встречаются в столбце B. Все узлы графа: `π_A(G) ∪ π_B(G)`. Узлы с входящими рёбрами: `π_B(G)`.

SQL:

```
SELECT A AS node FROM G
EXCEPT
SELECT B FROM G;
```

Это даёт узлы из `π_A(G) − π_B(G)`. Узлы, которые встречаются только в B, по определению имеют входящее ребро, поэтому они корректно исключаются.

Реляционная алгебра:

```
π_A(G) − π_B(G)
```

(b) Пары узлов (x, y), вместе покрывающие все узлы

Требуется: для каждого узла n в графе — либо (x, n) ∈ G, либо (y, n) ∈ G. Иначе: не существует узла n, такого что и (x, n) ∉ G, и (y, n) ∉ G.

SQL:

```
WITH nodes AS (
    SELECT A AS node FROM G
    UNION
    SELECT B AS node FROM G
)
SELECT x.node AS x, y.node AS y
FROM nodes x
CROSS JOIN nodes y
WHERE NOT EXISTS (
    SELECT 1 FROM nodes n
    WHERE NOT EXISTS (SELECT 1 FROM G WHERE A = x.node AND B
= n.node)
      AND NOT EXISTS (SELECT 1 FROM G WHERE A = y.node AND B
= n.node)
);
```

(c) Пары узлов (x, y) с расстоянием не более 4

Расстояние ≤ 4 означает существование пути длиной 1, 2, 3 или 4 рёбра.

SQL:

```
-- длина 1
SELECT A AS x, B AS y FROM G
UNION
-- длина 2
SELECT g1.A, g2.B
FROM G g1 JOIN G g2 ON g1.B = g2.A
UNION
-- длина 3
SELECT g1.A, g3.B
FROM G g1
JOIN G g2 ON g1.B = g2.A
JOIN G g3 ON g2.B = g3.A
```

```sql
UNION
-- длина 4
SELECT g1.A, g4.B
FROM G g1
JOIN G g2 ON g1.B = g2.A
JOIN G g3 ON g2.B = g3.A
JOIN G g4 ON g3.B = g4.A;
```

Реляционная алгебра — обозначим пути через соединение с переименованием:

```
G₁ = G
G₂ = π_{A,B}(ρ_{B→M}(G) ◁▷_{M=A} ρ_{A→M}(G))
G₃ = π_{A,B}(ρ_{B→M}(G) ◁▷_{M=A} G₂)
G₄ = π_{A,B}(ρ_{B→M}(G) ◁▷_{M=A} G₃)

Результат = G₁ ∪ G₂ ∪ G₃ ∪ G₄
```